Timing at 100 MHz: the failure, the cause and the fix

This page explains the timing failure seen after the first Vivado implementation of `zed2_top_standalone` (Way 1), what caused it, what was changed, how we know the change did not alter the behaviour, and what to do now.

1. What the report said

item	value	meaning
clock	100 MHz, period 10 ns (`zed2_standalone.xdc`)	every register-to-register path must settle within 10 ns
WNS (worst negative slack, setup)	**-0.687 ns**	the slowest path needs about 10.69 ns, so it is 0.687 ns too slow
failing paths	all inside `u_core/u_noc/g_node[..]/u_r` (the NoC routers)	from a FIFO read pointer `rptr_reg` to the output register `oflit_reg`, and to `cnt_reg`, `rptr_reg`, `cr_reg`
logic levels	about 12, with nets of fanout 37-43	long chain of logic plus big fan-out

What a negative WNS means. Static timing analysis adds up the delay of every gate and wire between two flip-flops and compares it with the clock period. With -0.687 ns the slowest path would only be safe up to about 1 / 10.69 ns = **93.5 MHz**. At 100 MHz a flip-flop at the end of that path could catch a value that has not settled yet. The design would probably still work on most boards at room temperature, but nobody can promise it. So a design with WNS < 0 is not "finished".

The other numbers in a timing summary:

name	what it is	must be
WNS / TNS	setup slack: the worst one / the sum of all negative ones	WNS >= 0, TNS = 0
WHS / THS	hold slack (a value must not change too *early*)	WHS >= 0, THS = 0
WPWS	pulse width slack (clock pins)	>= 0
Failing endpoints	how many flip-flops have a negative slack	0

When everything is met Vivado says *"All user specified timing constraints are met."*

2. Why the router path was too long

A router promises **2 clocks per hop**: in the first clock the flit is written into the input FIFO, in the second clock it is routed, arbitrated and copied into the output register. In the first version, all of the second clock's work hung off the FIFO memory:

```
rptr (FIFO read pointer, drives ~40 LUT-RAM address pins)       <- fan-out 37-43
  -> distributed-RAM read of the front flit (RAM32M, slow)
     -> dest field -> route table lookup -> "request" per output
        -> priority filter -> round-robin search from a 3-bit pointer (mod 5)
           -> grant -> 5-way output multiplexer -> oflit (output register)
           -> "taken" (compare grant with my port number) -> pop
              -> rptr, cnt, cr (the FIFO's own registers)
```

The distributed-RAM read plus the high-fan-out pointer alone cost about 2 ns, and everything else came after it, in the same clock.

3. The fix: same behaviour, shorter paths

The rule was: **no change in behaviour, not even by one clock**. The latency stays 2 clocks per hop, the arbitration order stays the same, and the tests and documents stay valid.

3.1 Router (`rtl/noc/noc_router.v`)

what	before	after
front flit	read from the FIFO memory every clock (`mem[rptr]`)	a register copy `front_q`, reloaded when the front leaves (from `mem[rptr+1]`, or straight from the input when the FIFO was nearly empty)
route of the front flit	looked up from its dest every clock	worked out once when the flit is **written**, stored next to it (`rmem`) and in `froute_q`, as a one-hot vector
"FIFO not empty", "input owns an output"	decoded from the counter / from the five output locks	registers `ne_q`, `busy_q`
round-robin	3-bit "last winner" pointer, search mod 5	a one-hot mask of the inputs *after* the last winner; grant = the first candidate in the mask, else the first candidate
output multiplexer and "taken"	binary select, compared with the port number	one-hot grant: AND-OR multiplexer, "taken" = go AND grant bit

The longest path now starts at a register (`front_q`, `froute_q`, `ne_q`, `busy_q`, the mask) and goes request -> priority -> grant -> output register or pop. The slow RAM read moved to the reload of `front_q`, which has the whole clock for itself.

3.2 Two other paths that were close

Once the router was fixed, a logic-depth estimate of the whole design showed two more paths close to the router's old length. They were shortened in the same way (same behaviour):

block	path	change
serial engine (`rtl/se/se_sm.v`)	program memory -> bit count n -> mask `(1 << n) - 1` (a 32-bit subtractor) -> shift registers / pins	the mask is built bit by bit (`bit i = i < n`): no subtractor
DMA writer (`rtl/hub/dma_writer.v`)	AXI write response -> `commit` -> `pending + commit` (32-bit adder) -> `>= BATCH` (32-bit comparator) -> `pending`, `age`, interrupt	both comparisons (`pending >= BATCH` and `pending + 1 >= BATCH`) are made from registers; the late `commit` signal only chooses one

3.3 Build scripts

`vivado/sources.tcl` now has `ssc2_impl_settings`, used by `build_standalone.tcl`, `build_ps_system.tcl` and `create_project.tcl`. It turns on physical optimisation after placement (directive *Explore*) and after routing. Both only act on paths that are short of time (for example, they copy a register that drives too many loads) and add a few minutes.

3.4 Estimated effect

Vivado is not available in this environment, so the numbers below come from yosys (`synth_xilinx`) with a simple delay model (LUT + wire 0.57 ns, MUXF7/F8 0.3 ns, CARRY4 0.25 ns, LUT-RAM read 1.4 ns). They are only good for comparing before and after; **the real result is your Vivado run**.

path (estimate, "ns" of the model)	before	after
router, worst path	8.6	5.2
serial engine, program memory -> registers / pins	9.1	7.8 (now the worst path of the controller)
DMA writer, write response -> batch counter	9.1	4.0

The model over-counts carry chains (a CARRY4 is much faster in a real FPGA), which is why the serial-engine and DMA paths score higher than the router before the fix although Vivado found the router path worst. The point of the table is the direction: the router path, which was Vivado's -0.687 ns, dropped by about 40 %, and the other two long paths dropped as well, so WNS should come out positive.

4. How we know nothing else changed

change	check	result
router	`sim/unit/tb_noc_router_equiv.v`: the new router and the old one (`sim/models/noc_router_ref.v`) get the same random inputs, side by side, at all six positions of the mesh; every output is compared at every clock. Random traffic with and without credit flow control, broken packets, FIFO overruns, prio, and resets in mid-traffic	0 differences (6 x 50 000 clocks in the test suite; 6 x 1 000 000 clocks in a longer run). A deliberately broken copy of the router fails at once.
router	the existing `tb_noc_router` (1547 checks, incl. 2 clocks per hop, fairness, prio, wormhole) and `tb_noc_mesh`	pass, unchanged
serial engine, DMA writer	formal equivalence proof with yosys (`equiv_make`, `equiv_induct`): old and new module are the same for **every** input sequence	proven (615 and 961 equivalence points). A deliberately broken copy is not proven, so the check works.
everything	14 unit testbenches, 3 system testbenches, `tb_zed2_gate` on the yosys gate-level netlist	all pass
everything	the printed output of every testbench (latencies, clock counts, packet counts, measured bit times) compared with the reference logs from before the change (`docs/sim_results/`)	identical, line for line

5. What it costs

Registers instead of memory reads cost flip-flops and a few LUTs. yosys (`synth_xilinx -flatten`, the whole Way 1 design `zed2_top_standalone`; LUTs include the LUTs used as distributed RAM, 4 per RAM32M / RAM64M):

	LUTs	flip-flops	LUT-RAM cells	block RAM
before	13 491	5 550	206	0
after	14 009 (+3.8 %)	6 320 (+13.9 %)	225	0

The extra flip-flops are mostly the front registers of the router inputs (about 40 bits per used router input: the front flit, its one-hot route and two flags); the extra LUTs mostly reload them. yosys estimates scatter by a few percent from run to run: the mesh alone grows by about 1 000 LUTs (3268 -> 4263) and 800 flip-flops, so for the whole design expect +4 % to +8 % LUTs and about +14 % flip-flops. yosys counts more LUTs than Vivado (your first Vivado run: 9 775 LUTs and 5 326 flip-flops), so in Vivado expect roughly **10 200-10 600 LUTs and about 6 100 flip-flops**: under a fifth of the Zynq-7020's LUTs and under 6 % of its flip-flops.

6. What to do now

1. Take the new files (new zip, or the branch `claude/wizardly-cori-h3t0hy`).
2. **Re-create the project**: double-click `windows\0_create_project_gui.bat` (it overwrites `build\gui`). If you keep your own project instead, the changed design files are `rtl/noc/noc_router.v`, `rtl/se/se_sm.v` and `rtl/hub/dma_writer.v`. If you added files with *Copy sources into project* ticked, Vivado uses its own old copies: remove them and add the new files without copying. `sim/models/noc_router_ref.v` and `sim/unit/tb_noc_router_equiv.v` are simulation sources only.
3. Run Synthesis -> Run Implementation -> Open Implemented Design -> **Report Timing Summary**. Expect WNS >= 0, TNS = 0, WHS >= 0, THS = 0 and "All user specified timing constraints are met". (Batch build: `windows\2_build_way1_fpga_only.bat` prints the WNS at the end.)
4. Optional: in the Sources window make `sim_noc_equiv` active and run its behavioural simulation (`ALL 18 CHECKS PASSED`), then switch back to `sim_1` and run the post-implementation simulation of `tb_zed2_gate` (`ALL 11 CHECKS PASSED`).
5. Write the new numbers into MEASUREMENTS.md section 5.

If WNS is still negative: open the timing report, click the worst path, and note its source, destination, levels and the split between logic and net delay. Then try *Implementation Settings -> Strategy -> Performance_ExplorePostRoutePhysOpt* and run implementation again. If it still fails, send the path details: the fix depends on which block it is in.

7. The DRC warning ZPS7-1

"[DRC ZPS7-1] PS7 block required" appears because Way 1 does not use the ARM side of the Zynq (no `processing_system7` block). For a design that is loaded over JTAG this is harmless: the bitstream is written and works. Way 2 has the PS7 in its block design, so the warning does not appear there.